

Chart Digitizer

Automated Milking Session Data Extraction System

Technical Explainer

University of Naples Federico II
MSc Data Science

Supervisor: Prof. Schiano Lo Moriello

June 2026

1. Project Overview

Chart Digitizer is an offline web application that automatically extracts curve data from Automated Milking System (AMS) PDF reports produced by a farming company. The system eliminates manual transcription of milking data, enabling direct digital analysis of milk flow and vacuum pressure measurements for each animal session.

Each PDF page contains **8 charts** arranged in a fixed 2-column × 4-row grid. Each chart represents one milking session for one animal and displays two curves:

- **Flusso** (blue) — milk flow rate in kg/min, Y-axis 0–9
- **Pressione assoluta** (green) — absolute vacuum pressure in 0.1 bar, Y-axis 0–9
- **X-axis** — milking duration in minutes, range 0–10

The pipeline outputs **one CSV file per chart**, named by animal ID and session date. Example: `animal_1001_2024-03-07.csv`. When two sessions for the same animal occur on the same date, a time suffix is appended: `animal_948_2024-03-07_0500.csv`.

2. System Architecture

The system uses a **classical Computer Vision pipeline** rather than deep learning. This decision was made because the PDF format is fixed and consistent, no large labeled dataset is available or required, the pipeline is fully explainable (satisfying academic requirements), and it runs entirely offline with zero internet dependency.

Pipeline Steps

1. **PDF Rasterisation** — PyMuPDF converts each PDF page to a high-resolution bitmap image at 150 DPI for image processing.
2. **Native Text Extraction** — PyMuPDF reads embedded text directly from the PDF (no OCR needed for real PDFs). Chart headers containing animal ID, date, and time are extracted with exact bounding-box positions.
3. **Accurate Cell Slicing** — Row boundaries are derived from the Y-coordinates of embedded text blocks, ensuring correct alignment even when chart heights vary. Each page is sliced into 8 individual chart cells.
4. **Per-Chart Processing** — For each cell: axis calibration via line detection, HSV colour isolation (blue channel for Flusso, green for Pressione), column-by-column pixel extraction, and coordinate mapping from pixel space to data space.
5. **CSV Export** — Pandas writes the extracted (time, value) pairs to a named CSV file. Two rows per time point: one for each curve.

Deployment Architecture

The system is packaged as a **self-hosted web application**. A Flask Python backend exposes a REST API; a lightweight HTML/CSS/JS frontend provides a drag-and-drop interface. The entire stack runs in a Docker container on a machine within the farm's local network. Farm staff visit a local URL (e.g. `http://milking.local`) from any browser on the network — no installation required, and no data leaves the premises.

3. Technology Stack

Layer	Tool / Library
PDF Rasterisation	PyMuPDF (fitz)

Metadata Extraction	PyMuPDF native text — no OCR for real PDFs
Image Processing	OpenCV (opencv-python)
OCR Fallback	Tesseract + pytesseract (PNG inputs)
Coordinate Mathematics	NumPy
CSV Export	Pandas
Synthetic Chart Generation	Matplotlib (PDF backend — embedded text)
Web Backend	Python Flask
Frontend	HTML / CSS / Vanilla JavaScript
Containerisation	Docker

4. Accuracy Results

The pipeline was validated against synthetic PDFs generated with a known ground truth. Synthetic charts mimic the real company PDF format exactly — same layout, embedded text headers, axis ranges, and curve shapes — allowing quantitative accuracy measurement. Every extracted curve is anchored at the plot origin ($t = 0$, value 0); the flow curve's return to zero is likewise extended to the end of the session window, recovering the segments that are drawn over the black axis lines and invisible to colour masking.

Metric	Result
Charts extracted	48 / 48 (100%)
Warnings	0
Flusso MAE — median	0.11 kg/min
Flusso MAE — mean	0.12 kg/min
Pressione MAE — median	0.20×0.1 bar
Pressione MAE — mean	0.19×0.1 bar
Flusso curve coverage	100.0 %
Pressione curve coverage	98.4 %

MAE = Mean Absolute Error versus known ground-truth coordinates.

5. Test Suite

Three synthetic PDF scenarios are generated by `synthetic_pdf_generator.py`, each with an accompanying JSON ground truth file containing exact curve coordinates:

File	Pages	Charts	Purpose
synthetic_1_baseline.pdf	1	8	Happy-path validation — normal sessions, typical curve shapes
synthetic_2_multipage.pdf	3	24	Multi-page iteration and CSV collision handling (time suffix)
synthetic_3_edge.pdf	2	16	Edge cases: flat curves, short sessions, duplicate animal IDs

Total

6

48

6. Deployment

The system is designed for **self-hosted, offline deployment** on the farm's local network. Data privacy is guaranteed — no animal data leaves the premises.

Deployment flow:

1. Docker container runs on a machine on the farm's network.
2. Farm staff open a browser and navigate to the assigned local URL (e.g. `http://milking.local` or the machine's IP).
3. User drags and drops the AMS PDF onto the upload area.
4. Pipeline runs server-side; progress is shown in real time.
5. CSV files appear as individual download links + a single ZIP download.

Local development access:

`http://localhost:5001` — launch with `./start.sh` from the project directory.

7. Project File Structure

File	Purpose
<code>synthetic/synthetic_pdf_generator.py</code>	Generates synthetic test PDFs with embedded text and ground truth JSON
<code>synthetic/synthetic_generator.py</code>	Legacy PNG generator (used for image-only testing)
<code>pipeline/pipeline.py</code>	Full CV extraction pipeline — rasterisation, slicing, colour isolation, CSV export
<code>tests/evaluate_accuracy.py</code>	Benchmarks pipeline output against ground truth JSON, reports MAE and coverage
<code>app.py</code>	Flask web backend — upload, job queue, status polling, CSV download endpoints
<code>frontend/index.html</code>	Single-file drag-and-drop web UI with real-time progress and file list
<code>Dockerfile</code>	Docker container definition for self-hosted deployment
<code>start.sh</code>	One-command local launch script

8. CSV Output Format

Each output CSV contains three columns. Rows are interleaved for both curves:

```
curve,time_min,value
Flusso_kg_min,0.043,0.000
Flusso_kg_min,0.115,0.177
Pressione_assoluta_0.1bar,0.115,9.000
Pressione_assoluta_0.1bar,0.231,8.943
```

Filename convention: `animal_{ID}_{YYYY-MM-DD}.csv` — or with time suffix if the same animal has multiple sessions on the same date: `animal_{ID}_{YYYY-MM-DD}_{HHMM}.csv`